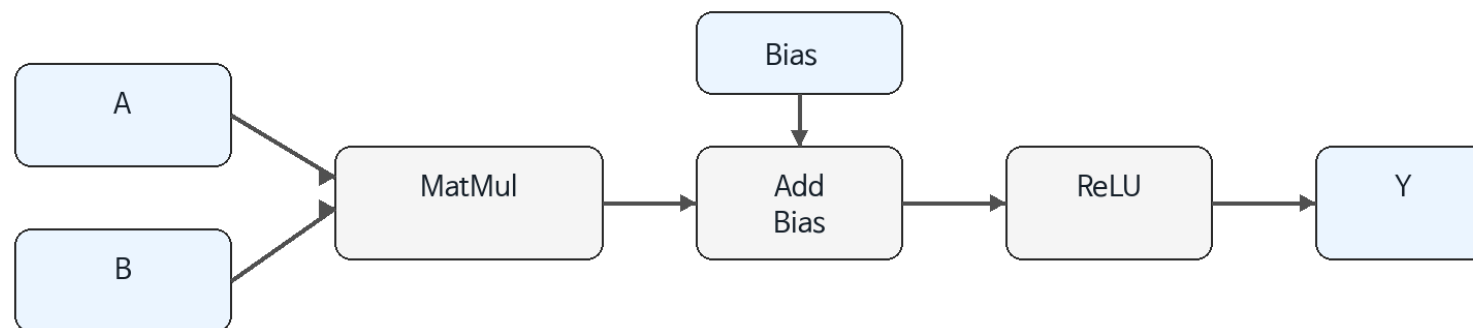


Lecture 10

Rewrite Pattern

MatMul + Add + ReLU Fusion Pattern Design for NPU Compiler

MatMul + Add + ReLU Fusion: Before / After



After rewrite:

```
npu_graph.matmul_epilogue {bias=true, activation="relu"}
```

One target-aware op preserves shape/type contract and exposes epilogue scheduling opportunity.

오늘의 위치

- 9 강 : Pass Infrastructure - pass 를 등록하고 pipeline 을 설계
- 10 강 : Rewrite Pattern - pass 내부에서 IR 을 실제로 바꾸는 최소 단위
- 11 강 : Dialect Conversion - legality 와 conversion target 기반 lowering

Learning Objectives

RewritePattern / OpRewritePattern / PatternRewriter 역할 구분
match phase 와 rewrite phase 를 분리해 안전한 pattern 작성
Greedy driver 의 worklist/fixpoint 동작 이해
MatMul+Bias+ReLU fusion contract 설계
FileCheck positive/negative regression test 작성

Why Rewrite Pattern Matters for NPU

NPU compiler 성능은 작은 graph rewrite 의 품질에 크게 의존
fusion 은 intermediate tensor lifetime 과 DRAM traffic 을 줄임
단 , target epilogue/quant/layout contract 를 증명하지 못하면 correctness bug
좋은 pattern 은 “왜 match 실패했는가” 까지 설명 가능

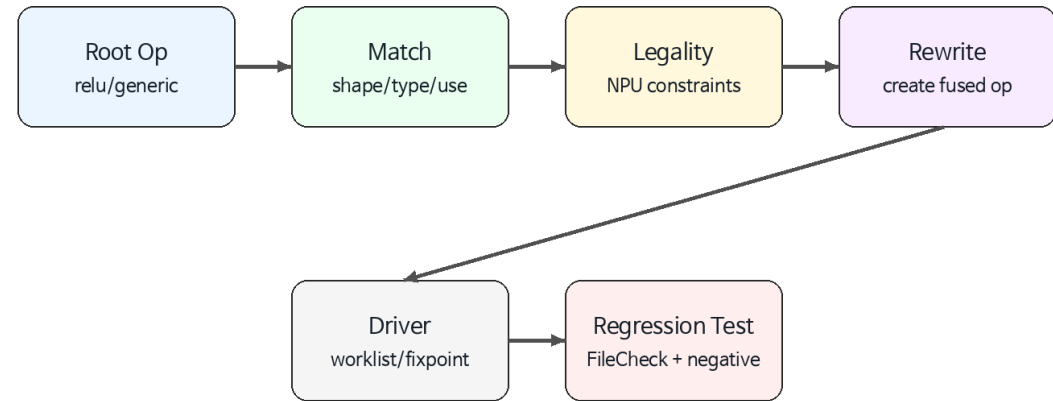
Pattern Rewriting Framework

Pattern Definition: root op, predicates, rewrite action, benefit

Pattern Application: driver, worklist, cost model, fixpoint

RewritePatternSet 에 pattern 을 모아 pass 에서 실행
NPU 예 : ReLU root 에서 AddBias 와 MatMul producer chain 을 역방향 match

MLIR Rewrite Pattern Flow for NPU Fusion



Rule: match first, mutate only through PatternRewriter, then verify the fused op contract.

RewritePattern Anatomy

Root operation name: 가능하면 명시

PatternBenefit: local priority signal

matchAndRewrite: match 와 mutation 을 한 함수에서 수행

Debug name/label: pattern filtering 과 debug log 에 필요

Bounded recursion: self-applicable pattern 이면 명시

PatternRewriter Rules

IR mutation 은 반드시 PatternRewriter API 로 수행
성공 전에는 IR 을 변경하지 않음
실패 시 notifyMatchFailure 로 이유를 남김
root op 는 replace / erase / in-place update 중 하나로 처리
direct erase 또는 raw mutation 은 driver state 를 깨뜨릴 수 있음

matchAndRewrite Discipline

1. root op 확인
2. producer/operand chain 확인
3. one-use / side-effect / dominance 확인
4. shape / dtype / layout / quantization 확인
5. target capability 확인
6. fused op 생성 후 replace/erase

Good vs Bad Rewrite

Good: 모든 조건이 증명된 뒤 `rewriter.create + replaceOp`

Good: 실패 사유가 구체적인 `notifyMatchFailure`

Bad: match 중간에 op 생성 후 실패

Bad: one-use 확인 없이 `producer erase`

Bad: target-specific fusion 을 canonicalization 에 숨김

Greedy Driver Overview

worklist 기반으로 op 를 방문

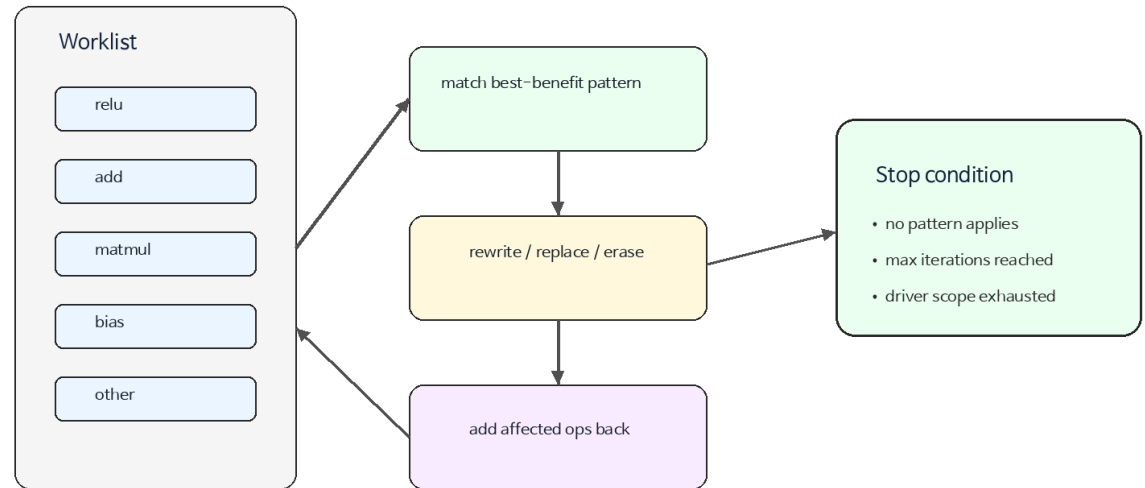
benefit 이 높은 pattern 을 지역적으로 적용

수정 / 생성된 op 는 다시 worklist 에 들어갈 수 있음

fixpoint 또는 iteration limit 에서 종료

cyclic rewrite 는 bug 로 취급해야 함

Greedy Rewrite Driver: Worklist and Fixpoint



Pattern Benefit Strategy

더 구체적인 pattern 에 더 높은 benefit

MatMul+Bias+ReLU > MatMul+Bias > Add identity

benefit 은 target legality 를 대체하지 않음

낮은 benefit 이라도 canonicalization 이 먼저 필요할 수 있음

Canonicalization vs Optimization

Canonicalization: IR 을 단순하고 안정적인 형태로 정리

Optimization: target contract 와 성능 목표를 반영

Pass correctness 는 canonicalize 존재에 의존하면 안 됨

NPU fusion 은 별도 optimization pass 로 두는 것을 권장

Fusion Target: MatMul + Add + ReLU

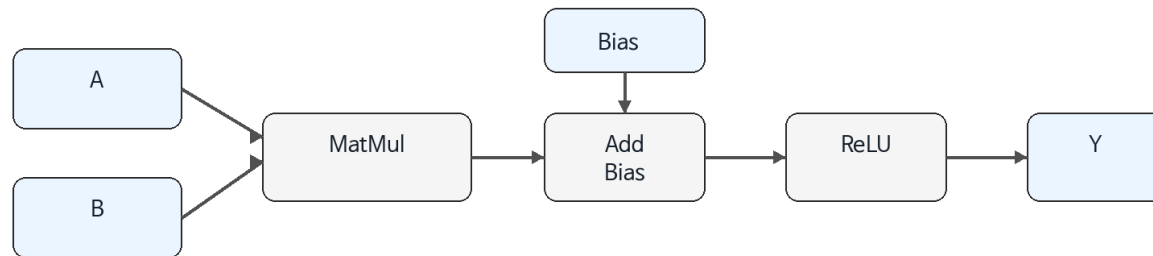
Input: MatMul result 에 bias add 후 ReLU 적용

Output: npu_graph.matmul_epilogue

Benefit: intermediate tensor 제거 , epilogue scheduling, DRAM traffic 감소

Risk: broadcast/quant/layout/one-use 조건 미확인 시 correctness bug

MatMul + Add + ReLU Fusion: Before / After



After rewrite:

```
npu_graph.matmul_epilogue {bias=true, activation="relu"}
```

One target-aware op preserves shape/type contract and exposes epilogue scheduling opportunity.

Root Op 선택

권장 root: ReLU output op

이유 : final result 를 replacement 결과로 바로 대체 가능

producer chain: ReLU -> AddBias -> MatMul

consumer 기준으로 pattern 전체를 닫기 쉬움

Matcher Decision Tree

root ReLU 확인

producer AddBias 확인

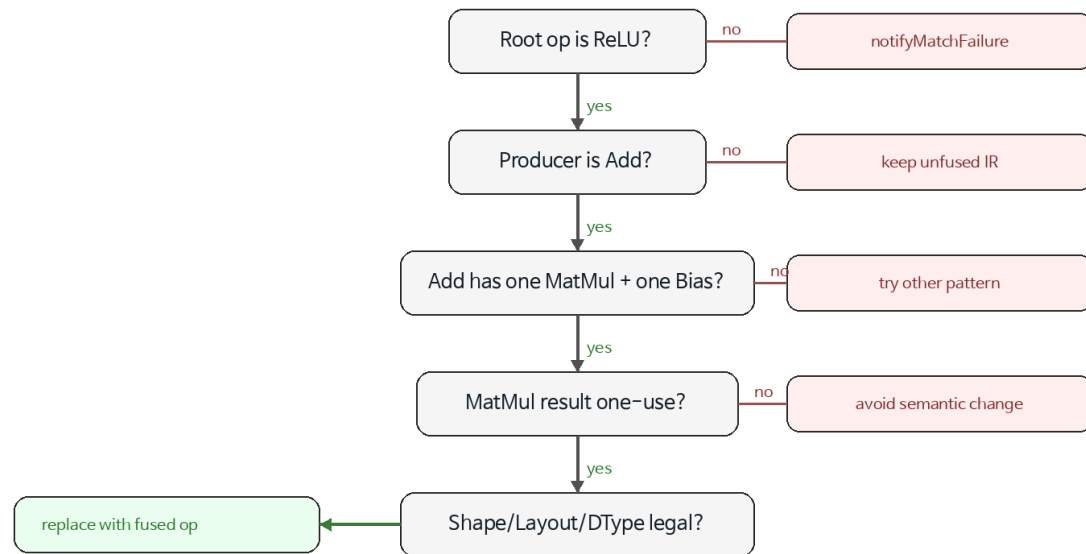
MatMul + Bias operand classification

producer one-use 확인

shape/layout/dtype target legality 확인

모든 조건이 통과해야 rewrite

Matcher Decision Tree: Fuse or Reject



Semantic Checks

ReLU 가 정확히 $\max(x, 0)$ 인지 확인

Add 가 bias broadcast 인지 elementwise residual add 인지 구분

MatMul contraction semantics 가 target kernel 과 일치

side-effecting op 또는 ordering dependency 없음

Use-Def Checks

MatMul result one-use

AddBias result one-use

replacement result type equals ReLU result type

shared producer 는 negative test 로 보호

clone-based fusion 은 별도 cost model 과 semantic proof 필요

Shape / Layout Checks

bias shape: scalar, [N], 또는 target-supported broadcast

output rank 와 static/dynamic dim policy

lhs/rhs transpose 또는 packed layout attribute

layout transform cost 가 fusion benefit 을 상쇄하지 않는지 확인

DType / Quantization Checks

FP32/BF16/FP16 path: activation 위치 비교적 단순

INT8/INT4 path: accumulator, requant, saturation, clamp 위치 중요

per-tensor/per-channel scale support 확인

activation 이 requant 전 / 후 어디에 들어가는지 ABI 와 kernel 에 맞춤

NPU Fusion Contract

fusion pattern 은 target contract 를 증명하는 작은 verifier

증명 실패 시 unfused IR 유지가 안전한 기본값

diagnostic reason 은 compiler tuning 의 핵심 데이터
contract 는 op verifier 와 pass test 로 중복 방어

NPU Fusion Contract: What the Pattern Must Prove

Semantic	Shape/Layout	DType/Quant	Use/Memory	Target
MatMul + bias + relu exactly preserves math	Output rank/static dims, bias broadcast legal	Accumulator, requant, saturation policy legal	MatMul/add/relu producers not shared unsafely	Epilogue supported by target kernel
No hidden side effects or ordering dependency	Layout transform cost acceptable	INT8/INT4 scale/zero-point compatible	Fusion reduces intermediate tensor lifetime	SRAM footprint within tile budget

Outcome: use notifyMatchFailure for every failed proof; keep unfused IR if a contract cannot be proven.

Replacement Op Design

`npu_graph.matmul_epilogue(lhs, rhs, bias)`

attributes: activation, layout, accumulator, quant policy

result type: original ReLU output type

location: fusedLoc(matmul, add, relu)

later lowering: `npu_kernel.matmul` + DMA/barrier + runtime descriptor

C++ Pattern Skeleton

```
struct FuseMatMulBiasReluPattern
: OpRewritePattern<linalg::GenericOp> {
LogicalResult matchAndRewrite(linalg::GenericOp relu,
                             PatternRewriter &rewriter) const override {
    if (!isReluGeneric(relu))
        return rewriter.notifyMatchFailure(relu, "not relu");
    auto add = getSingleProducerOfType<linalg::GenericOp>(relu.getInput());
    if (!add || !isAddBiasGeneric(add))
        return rewriter.notifyMatchFailure(relu, "not add bias");
    if (!matmul->hasOneUse() || !add->hasOneUse())
        return rewriter.notifyMatchFailure(relu, "shared producer");
    auto fused = rewriter.create<npu_graph::MatmulEpilogueOp>(...);
    rewriter.replaceOp(relu, fused.getResults());
    rewriter.eraseOp(add);
    rewriter.eraseOp(matmul);
    return success();
}
};
```

Failure Diagnostics

```
notifyMatchFailure(relu, "root is not ReLU")  
notifyMatchFailure(add, "cannot classify matmul and bias")  
notifyMatchFailure(relu, "producer result is shared")  
notifyMatchFailure(relu, "unsupported bias broadcast")  
notifyMatchFailure(relu, "quantization policy not representable")
```

Pass Skeleton

```
void runOnOperation() override {  
    RewritePatternSet patterns(&getContext());  
    patterns.add<FuseMatMulBiasReluPattern>(&getContext());  
  
    GreedyRewriteConfig config;  
    config.useTopDownTraversal = true;  
    config.maxIterations = 10;  
  
    if (failed(applyPatternsGreedily(getOperation(),  
                                    std::move(patterns), config)))  
        signalPassFailure();  
}
```


PDL / Declarative Matching

PDL can describe graph-shaped pattern matching declaratively

Native constraints can handle target legality predicates

C++ rewrite callback can create fused target op

Useful for prototyping and pattern library readability

Testing Strategy

Positive: fused op appears, old ops disappear

Negative: shared producer, unsupported bias, unsupported dtype do not fuse

Diagnostic: failure reason is stable enough for debugging

Stress: dynamic shape, multi-user, layout mismatch, quant mismatch

FileCheck Template

```
// RUN: mlir-opt %s --npu-fuse-epilogue | FileCheck %s
// CHECK-LABEL: func.func @matmul_bias_relu
// CHECK: "npu_graph.matmul_epilogue"
// CHECK-SAME: activation = "relu"
// CHECK-NOT: linalg.generic

// Negative case:
// CHECK-LABEL: func.func @shared_matmul_result_must_not_fuse
// CHECK-NOT: "npu_graph.matmul_epilogue"
```

Debug Recipe

Run pass with `--mlir-print-ir-after-change`

Use `-debug-only=greedy-rewriter` when available

Use pattern debug labels to filter

Reduce test with specific positive/negative examples

Add verifier checks to fused op

Integration in NPU Pipeline

After canonicalize/cse and graph cleanup

Before bufferization and target memory scheduling

Before layout is fully frozen, but after enough shape inference

Follow with npu_graph legality check and cost model pass

Common Mistakes

one-use omission

broadcast semantics confusion

dtype/quant clamp order mismatch

cyclic rewrite via inverse patterns

pattern benefit used as a substitute for legality

only positive tests

Exercise A

Annotate root/producers/operands in 01_linalg_matmul_add_relu_before.mlir

Mark which op is replaced and which ops are erased

Write 5 notifyMatchFailure reasons

Exercise B

Complete C++ helper checklist

isReluGeneric

isAddBiasGeneric

isMatMulGeneric

classifyAddInputs

checkShapeDTypeLayoutForTarget

Exercise C

Design test matrix

Positive: fp32, bias vector, supported layout

Positive: int8 with legal requant

Negative: shared MatMul result

Negative: unsupported broadcast/layout/dtype

Homework

Submit 1-page Fusion Pattern Spec
Complete pass skeleton with pattern registration
Write 3 positive and 5 negative FileCheck tests
Define runtime descriptor fields for fused epilogue

Takeaways

Rewrite pattern is the smallest executable compiler optimization unit

Safe matching is more important than aggressive fusion

NPU fusion must prove semantic + target legality

Debuggability comes from good failure reasons and negative tests

Next: Dialect Conversion and legal op sets

References

MLIR Pattern Rewriter documentation
MLIR Canonicalization documentation
MLIR Dialect Conversion documentation
MLIR Quickstart Rewrites tutorial
MLIR Greedy Pattern Rewrite Driver API